

AstraNotes

Week 5.2 — Requirements-to-UML Traceability Matrix

Jonathan Fong · CSEN 296B · Prof. Paddu Melanahalli

1. Requirements-to-UML Traceability Matrix

Seven requirements were selected spanning functional, non-functional, and security categories. Evidence is drawn from the Week 4.1 UML Design Package (class diagram, object diagram, use case diagram, activity diagram, deployment diagram). Requirements are taken verbatim from the Week 3.1 Refined Requirement Baseline.

Status legend:

Fully Traced	Partially Traced	Weakly Traced	Not Traced
--------------	------------------	---------------	------------

Req ID	Requirement Statement (Refined)	Class / Object Evidence	Use Case / Activity Evidence	Deployment Evidence	Status	Gap Note
FR-1	The system shall create a TextNote, VoiceNote, or SecureNote identified by a UUID (RFC 4122 v4) and a user-supplied title of 1–255 non-whitespace characters. Creation is performed exclusively through NoteFactory.	NoteFactory (create(), reconstructRecord()), Note hierarchy (TextNote, VoiceNote, SecureNote), NoteManager.add(); object diagram shows factory reference on manager.	Create Note use case includes Choose Note Type; NoteFactory lane in activity diagram allocates unique_ptr, generates UUID, and stamps timestamps; validation gates shown.	NoteFactory appears inside the AstraNotes Process component in the deployment diagram.	Fully Traced	None. All three diagram types provide consistent, specific evidence.
FR-3	The system shall delete a note by UUID. Deletion removes the note and releases all resources. For SecureNote, key material and decryption buffers shall be zeroed before deallocation. A missing UUID returns a failure status without throwing.	NoteManager.remove(id), map<UUID, unique_ptr<Note>> (auto-dealloc via RAII). SecureNote ciphertext_ field implies zeroing on destruct.	Delete Note use case includes Persist Notes to Disk. No activity diagram covers the delete workflow — only SecureNote creation is diagrammed.	No explicit deployment evidence; deletion is an in-process operation with no filesystem artifact beyond the updated notes.json.	Partially Traced	Activity diagram covers only SecureNote creation; a delete workflow with the SecureNote-zeroing step and missing-UUID path is absent.
FR-4 / FR-4b	The system shall persist all notes to a single JSON file. On startup each note is reconstructed via NoteFactory. Corrupt files are renamed to <name>_quarantine.<timestamp> and valid records still load.	FileStorage (save(), load(), remove()), StorageInterface (pure virtual), NoteFactory.reconstructRecord(); object diagram shows storage reference on manager.	Load Notes from Disk included by View Note; Persist Notes to Disk included by Create Note, Edit TextNote, Delete Note. Activity diagram shows atomic write and rollback on write failure.	notes.json artifact in the User Filesystem node; FileStorage component inside the AstraNotes Process; POSIX streams	Fully Traced	Quarantine rename (FR-4b) appears only in the requirement text; no diagram explicitly depicts the quarantine branch. Minor gap — all other evidence is complete.

Req ID	Requirement Statement (Refined)	Class / Object Evidence	Use Case / Activity Evidence	Deployment Evidence	Status	Gap Note
				connectors label the I/O path.		
FR-5	The system shall allow reading a SecureNote only after the correct passphrase (≥8 chars). Plaintext shall not be retained in any data member, log, or buffer after access; the decryption buffer shall be zeroed immediately after use.	SecureNote (ciphertext_ : bytes, unlock(), lockContent()), EncryptionEngine interface (inject), AESEngine concrete class. Object diagram shows ciphertext = <0x4F2A...> at rest.	Unlock SecureNote extends View Note with guard [note is SecureNote]; Encrypt Content extends Create Note with guard [type = SecureNote]. Activity diagram shows plaintext discard step annotated SPR-1.	AESEngine linked via OpenSSL libcrypto in the C++17 Runtime node; EncryptionEngine annotated SPR-2 (mockable); notes.json annotation confirms ciphertext-only.	Fully Traced	<i>Passphrase minimum-length constraint (FR-5) and typed error codes (FR-5a) are not visible in any diagram — they exist only in the requirement text.</i>
FR-6	The system shall search the in-memory note collection by title substring. Search shall be case-insensitive. No file I/O shall occur during search. An empty query returns the full collection.	NoteManager.searchByTitle(query: string). No dedicated class or data structure is shown — search is a single method on the controller. Class diagram note confirms FR-7 annotation.	Search Notes by Title use case annotated 'FR-7: title-substring, case-insensitive, in-memory only.' No activity diagram for this workflow.	No deployment evidence needed; search is purely in-process.	Partially Traced	<i>Case-insensitivity, empty-query, and no-I/O behaviors are stated in the requirement but not demonstrated in any diagram. A sequence or activity diagram for the search path would close the gap.</i>
NFR-1	UUID-keyed note lookup shall complete in O(1) average time using an unordered_map or equivalent hash structure. The data structure shall be the sole authoritative in-memory store.	NoteManager.notes_ declared as map<UUID, unique_ptr<Note>> in the class diagram with annotation 'NFR-1: O(1) lookup via unordered_map keyed by UUID.' Week 8.1 fix/test-cleanup branch corrected this to unordered_map in the implementation.	Indirectly supported: all state-changing use cases (Create, Edit, Delete) route through NoteManager, implying a single authoritative store. Not explicitly modeled in behavioral diagrams.	No deployment evidence; lookup performance is an in-process runtime property.	Partially Traced	<i>Class diagram labels the field as map<> (not unordered_map<>), creating a discrepancy with the NFR text. The implementation was corrected in Week 8.1 but the diagram was not updated.</i>
SPR-1	No SecureNote plaintext shall appear in any JSON file, log, core dump, or temporary file. SecureNote JSON records shall contain only ciphertext, the UUID, the title, and the created/modified timestamps.	SecureNote.ciphertext_ declared as bytes (not string). Object diagram annotation: 'ciphertext = <0x4F2A...> — bytes, not a readable string. No plaintext held in memory at rest.'	Activity diagram shows 'Discard plaintext from memory' step after EncryptionEngine.encrypt() returns, directly annotated SPR-1. Create Note use case extends Encrypt Content before persistence.	notes.json artifact annotated 'SecureNote: ciphertext only (SPR-1).' GTest Runner deployment note confirms EncryptionEngine is mocked in EncryptionTests.	Fully Traced	<i>Log file and core dump exclusions (added in refined baseline) appear only in the requirement text. The plaintext-discard enforcement path is otherwise well evidenced across all diagram types.</i>

2. Traceability Metrics Summary

Metric	Count
Total requirements reviewed	7
Fully Traced	4
Partially Traced	3
Weakly Traced	0
Not Traced	0
Requirements with no diagram gap	4
Requirements with at least one gap	3
Major UML elements without clear requirement reason	1

The one UML element without a clear requirement reason is VersionHistory / VersionEntry. Both classes appear in the class diagram (Figure 1) and object diagram (Figure 2) with a fully defined interface, but no requirement in the initial or refined baseline explicitly mandates version history. The closest charter item is the vague client request "I need to see a history of my changes so I don't lose work," but this was never translated into a formal FR, NFR, or SPR with acceptance criteria. The View Version History use case exists in Figure 3 but maps to no user story (US-01 through US-06). This is the primary over-design signal in the current package.

3. Gap Analysis

Three requirements are partially traced; each has a specific, actionable gap that should be addressed before implementation begins.

FR-3 — Delete Note

The delete workflow has structural support in NoteManager.remove() and RAII deallocation via unique_ptr, but no behavioral diagram covers it. The activity diagram documents only SecureNote creation. Before implementing deletion, a sequence or activity diagram should show: (1) the UUID-lookup step, (2) the missing-UUID failure path returning a status without throwing, and (3) the SecureNote-specific zeroing of key material before deallocation (required by the refined baseline). Without this, the SecureNote zeroing step has no enforcement checkpoint and is easily missed during code review.

FR-6 — Title Substring Search

The use case diagram annotates search as case-insensitive and in-memory (FR-7 annotation), but the three behavioral constraints — case-insensitivity, empty-query returning the full collection, and the no-file-I/O guarantee — are invisible in any diagram. The Week 8.1 implementation correctly handles all three, but a

reviewer reading the design package alone cannot verify it. A short sequence diagram or annotated activity diagram showing the query transformation and the strict mock StorageInterface (confirming no I/O) would make SPR-2's mockability requirement visible and give the test set a clear design anchor.

NFR-1 — $O(1)$ UUID Lookup

The class diagram labels `NoteManager.notes_` as `map<UUID, unique_ptr<Note>>`, which is $O(\log n)$, not $O(1)$. The Week 8.1 fix/test-cleanup branch corrected the implementation to `unordered_map`, but the class diagram was not updated. This is a direct inconsistency between the design artifact and the codebase. The class diagram should be updated before implementation continues so the diagram remains the reliable source of truth. The NFR text and the inline annotation both specify `unordered_map`; the diagram field declaration should match.

VersionHistory — Overdesign Risk

`VersionHistory` and `VersionEntry` are fully elaborated in the class and object diagrams but trace to no formal requirement. The View Version History use case is included in Figure 3 with no corresponding user story or FR/NFR/SPR. Before Sprint 3, this element should either be promoted to a formal requirement (with acceptance criteria) or flagged as out-of-scope for v1.0 and removed from the active class diagram to reduce implementation surface. Building and testing `VersionHistory` without a requirement baseline creates untraceable scope.

4. AI Use Note

AI was used to help structure the initial matrix layout and suggest which requirements to include given the assignment's instruction to select the five to eight most important. The matrix content — the specific diagram evidence, status labels, and gap notes — reflects my own judgment after working through each diagram against the requirement text. AI helped identify the `VersionHistory` overdesign signal by prompting a systematic check of every named class against the requirement table. The gap analysis narrative is my own: the three partially-traced items and the `VersionHistory` concern were identified during the matrix build, not generated independently.